UNIT -I

- UML-- Unified Modeling Language (UML)
- It is a general-purpose visual modeling language for systems.
- It has inbuilt extensibility mechanisms.
- It is associated with modeling OO software systems
- UML does not give us any kind of modeling methodology.
- It is not tied to any specific methodology or life cycle.
- UML is the open, Industry-standard visual modeling language approved by the OMG

What is UML, UP?

- The Unified Process (UP) is a methodology.
- UP uses UML as underlying visual model syntax .
- UML it self can provide the visual model support for other methods.
- OPEN – Object Oriented Process, Environment, and Notation

Goal of UML &UP

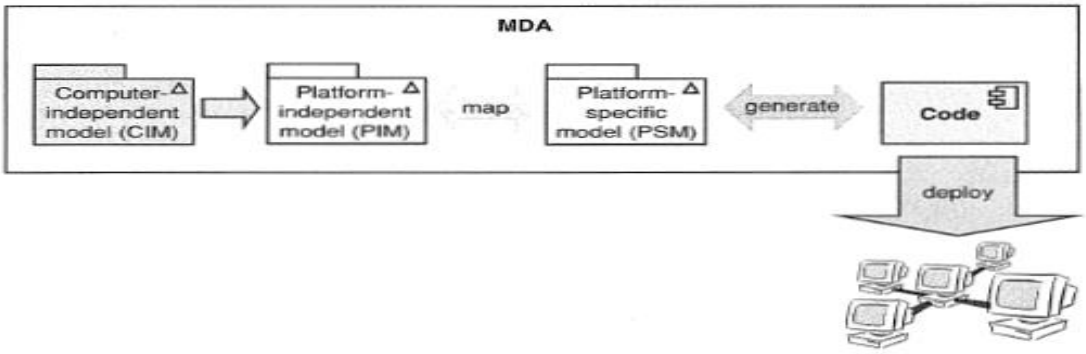
- To support and encapsulate best practice in software engineering based.

MDA:

- The future of UML defined by a recent OMG initiative called Model Driven Architecture (MDA).
- MDA defines a vision for how software can be developed based on models.
- Uses a MDA modeling tool.

Vision:

- The models drive the production of the executable software architecture.



MDA- the future of UML

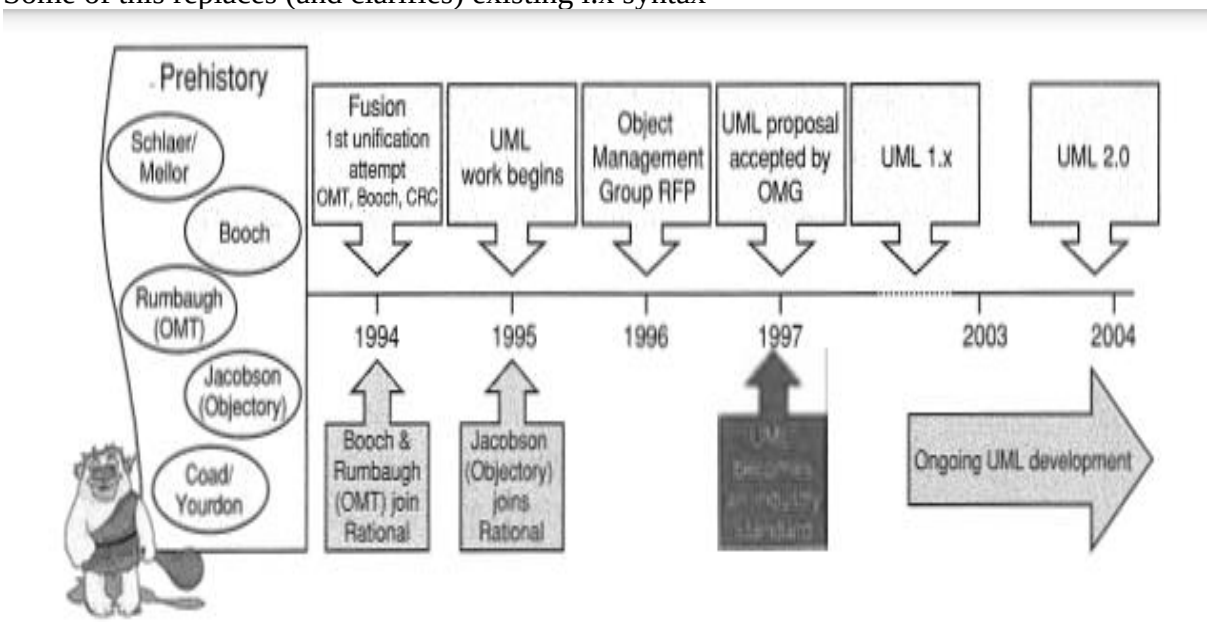
- CIM
  - It is a model at a very high level of abstraction
  - captures key requirements of the system
  - captures key vocabulary of the problem domain – those are *independent* of computers.
- PIM
  - It is a model that expresses the business semantics of the software system independently of any platform
  - Different MDA tools support different levels of platform independence
- PSM
  - Source code is generated from the PSM against the target platform
- Ex MDA tools: iUML, ArcStyler, Eclipse Modeling Framework, AndroMDA

Why Unified, Objectives?

- UML unification is riot just historical in scope.
- UML attempts across several different domains.
- Development life cycle -- visual syntax for modelling (SDLC based requirement gathering to implementation)
- Application domains -- Embedded systems to management decision support systems.
- Implementation languages and platforms -- language neutral and platform neutral
- Development processes – UML support many other software engineering processes.
- Its own internal concepts -- consistent and uniform

The Birth of UML

- Prior to 1994, several competing visual modeling languages and methodologies existed.
- All with their strengths and weaknesses.
- The clear leaders were
  - Booch (the Booch Method) and
  - Rumbaugh (Object Modeling Technique or OMT), Joined Rational Corporation
  - Methodologies side, Jacobson
- In 1996 – OMG (Object Management Group)– RFP. 1997 -industry-standard OO visual modeling language was born.
- In 2000, , UML 1.4 introduced a significant extension to UML by the addition of action semantics.
- UML 2 makes many syntactic changes to UML compared to UML l .x
- Introduces a lot of new visual syntax.
- Some of this replaces (and clarifies) existing l.x syntax



Objects and UML

UML models the world as systems of interacting objects. An object is a cohesive cluster of data and function.

- Model software and other systems as *collections of interacting objects.*
- Fit with OO software systems and languages.
- works well for business processes and other applications.

There are two aspects to a UML model.

- Static structure: Describes what types of objects are important for modeling the system and how they are related.

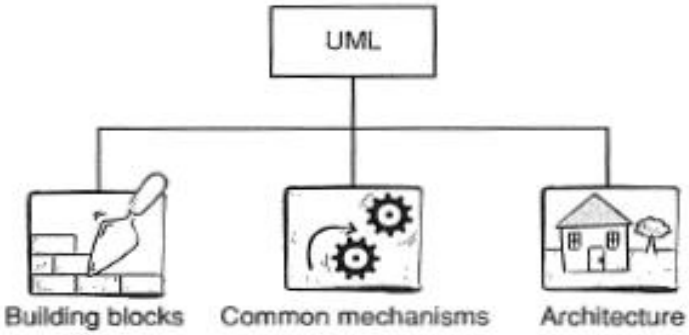
- Dynamic behaviour: Describes the life cycles of these objects and how they interact with each other to deliver the required system functionality.

UML Structure

It means how UML works as a visual language.

This structure consists of

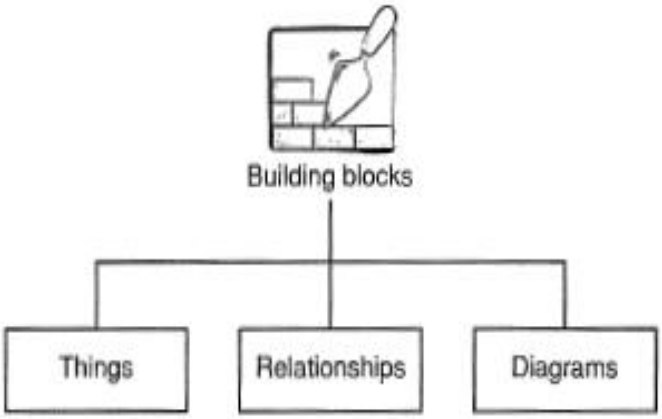
- Building blocks -These are the basic UML modeling elements, relationships, and diagrams;
- Common mechanisms - Common UML ways of achieving specific goals;
- Architecture - The UML view of system architecture.



UML building blocks:

UML is composed of just three building blocks.

- Things - The modeling elements themselves.
- Relationships –
  - Tie things together.
  - Specify how two or more things are semantically related.
- Diagrams –
  - *Views* into UML models.
  - Show collections of things that "tell a story" about the software system.
  - Way of visualizing *what* the system will do (analysis-level diagrams) or *how* it will do it (design-level diagrams).



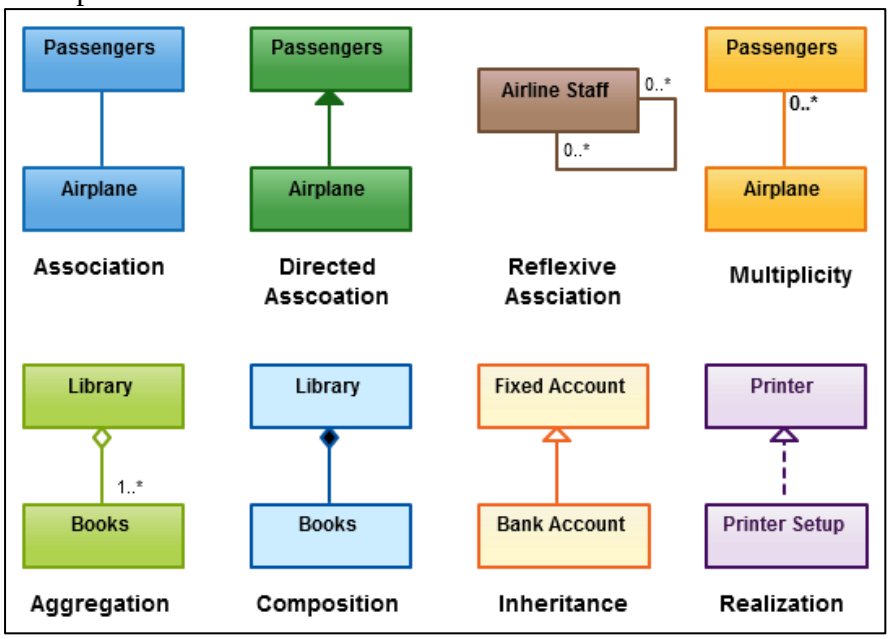
Things :

- Structural things - The Nouns of a UML model,
  - Interface, collaboration, use case, active class, component, node;
- Behavioral things - The Verbs of a UML model
  - Interactions, activities, state machines;
- Grouping things - The Package,
  - Which is used to group semantically related modeling elements into cohesive units;
- Annotational things - The Note,
  - Which may be appended to the model to capture ad hoc information, very much like a yellow sticky note.

Relationships

Type of relationship	UML syntax source target	Brief comantice
Dependency	---------->	The source element depends on the target element and may be affected by changes to it
Association	————	The description of a set of links between objects
Aggregation	◇————	The target element is a part of the source element
Composition	◆————	A strong (more constrained) form of aggregation
Containment	⊕————	The source element contains the target element
Generalization	————▷	The source element is a specialization of the more general target element and may be substituted for it
Realization	----------▷	The source element guarantees to carry out the contract specified by the target element

Examples:



Diagrams:

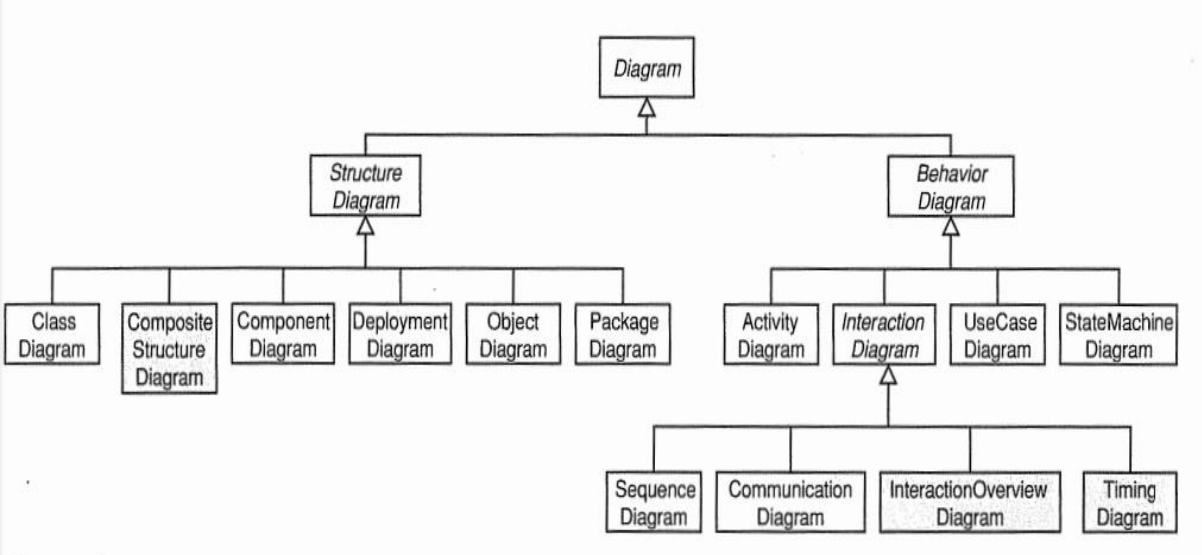


Figure 1.6

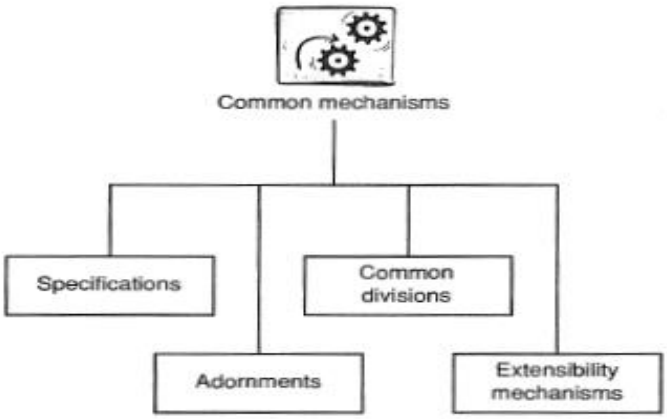
Rules:

UML has semantic rules for the following −

- Names
- Scope
- Visibility
- Integrity
- Execution

common mechanisms:

UML has four common mechanisms:

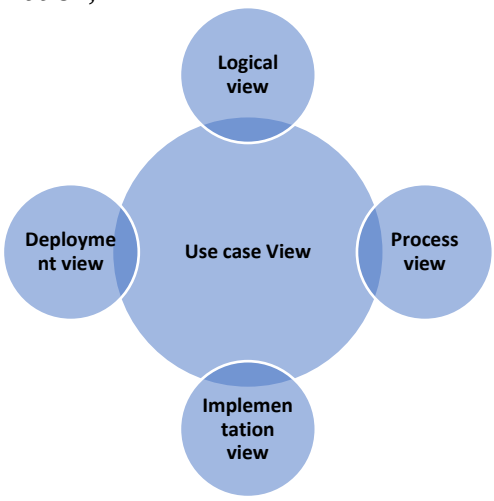


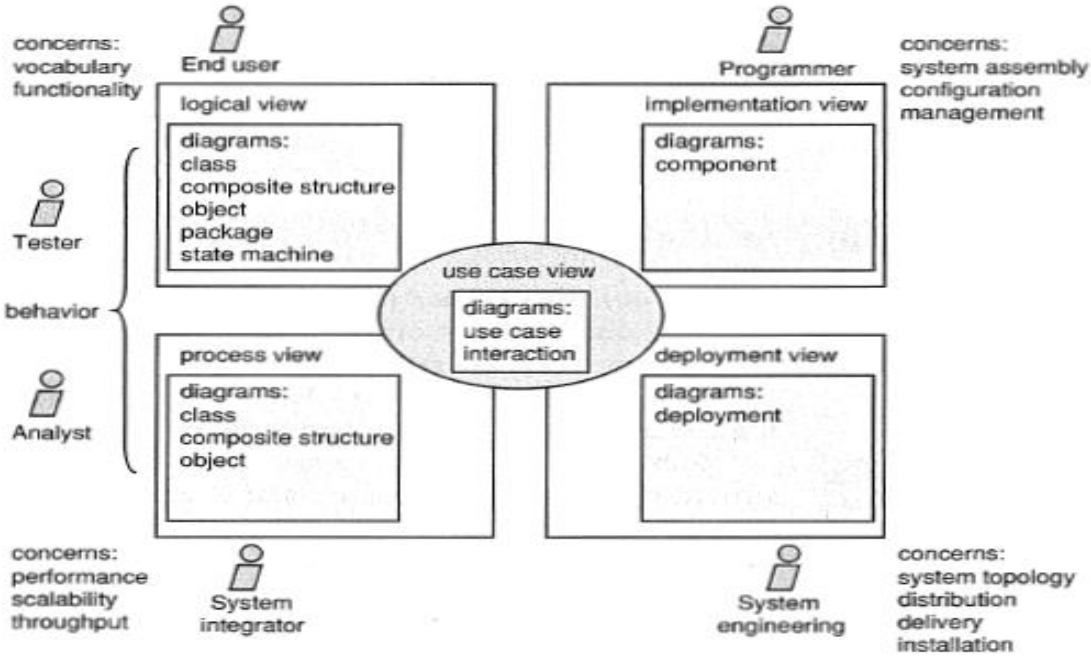
- Specifications –
  - Textual descriptions of the features and semantics of model elements - the meat of the model;
- Adornments –
  - Each element in UML has a unique graphical notation.
  - Besides, there are notations to represent the important aspects of an element like name, scope, visibility, etc.
- Common divisions: (OOAD can be divided in many ways)
  - -classifier and instance:

        - - Classifier - The abstract notion of a type of thing, e.g., a bank account;
        - - Instance - a specific instance of a type of thing, e.g., my bank account;
    - - Interface and Implementation:
        - - Interface - a contract that specifies the behavior of a thing;
        - - Implementation - the specific details of how the thing works;
- Extensibility mechanisms:
    - It is a collection of constraints, stereotypes, and tagged values.
    - It allows you to customize UML for a specific purpose.
    - Constraints: It extends the semantics of UML building blocks.
        - Allow us to add new rules to modeling elements;
    - Stereotypes (<< >>): It extends the vocabulary of the UML building blocks
        - Introduce new modeling elements based on old ones;
    - Tagged values : It extends the properties of UML building blocks.
        - Allow us to add new properties to model elements
        - Tagged value is a keyword with an associated value.

UML System Architecture (4+1)

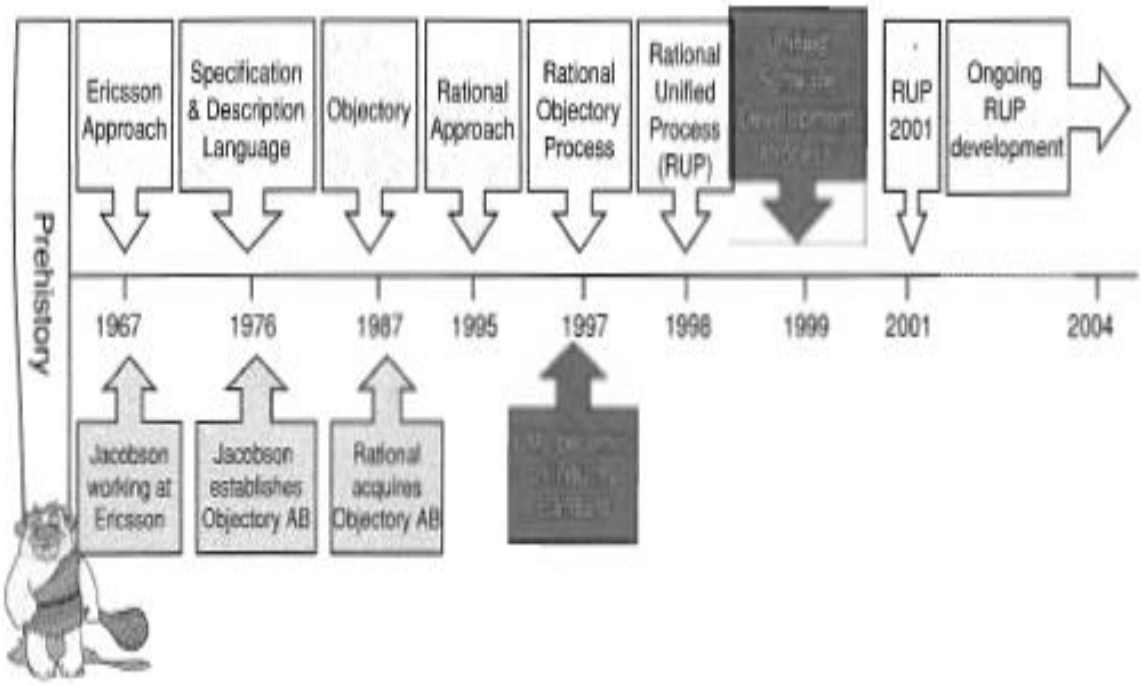
- Usecase View – It describes all views are united by the use case view, which describes stakeholder requirements.
- Logical view – It describes system functionality and vocabulary;
- Process view – It describes system performance, scalability, and Throughput;
- Implementation view – It describes System assembly and Configuration management;
- Deployment view – It describes System topology, Distribution, Delivery, and Iinstallation;

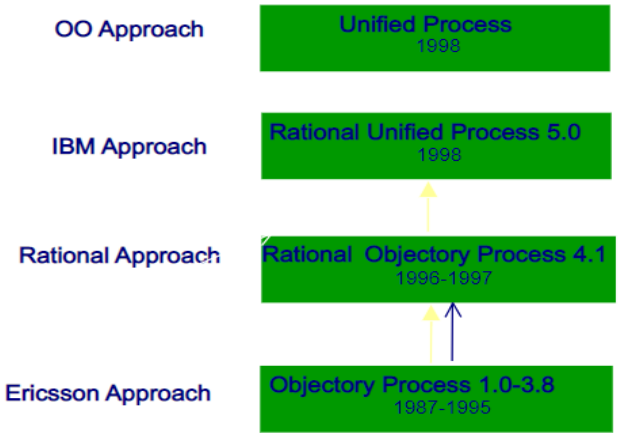




2. What is UP?

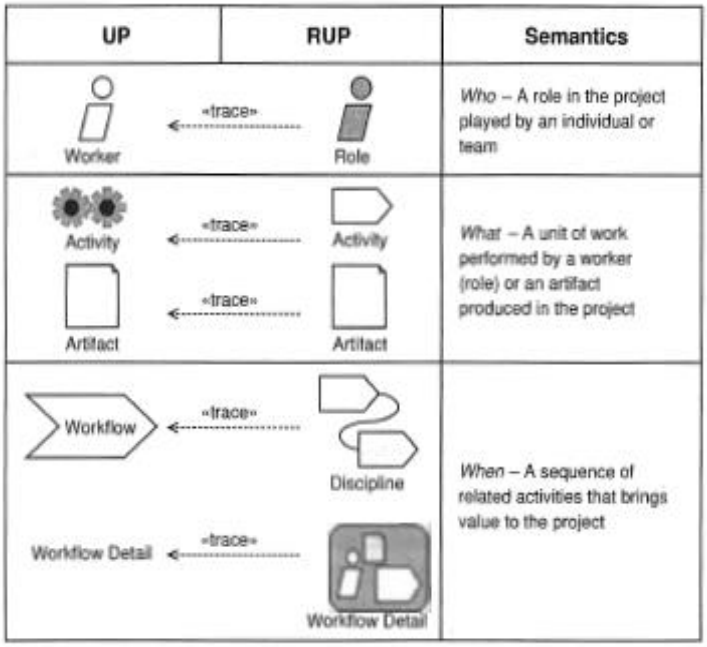
- A software engineering process (SEP), also known as a software development Process. SEP → SDP
- It defines the *who, what, when,* and *how* of developing software.
- A SEP describes how requirements are turned into software.
- The Unified Software Development Process (USDP) is a SEP from the authors of the UML.
- It is commonly referred to as the Unified Process or UP. USDP → UP
- The UML project meant to provide both a Visual language *and* a SEP
- UP began in 1967 at Ericsson.
- UP is a mature, open SEP from the authors of UML.





What is RUP?

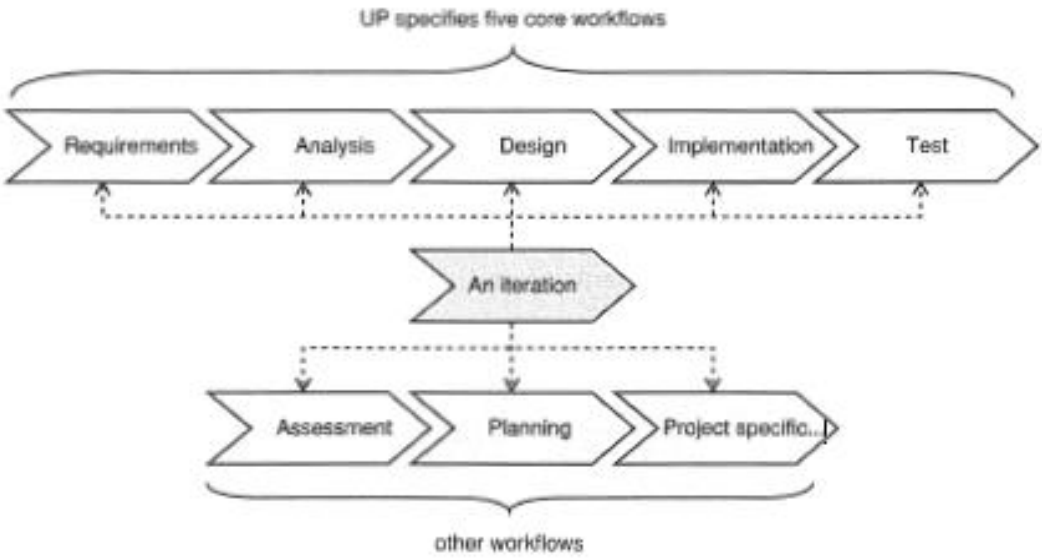
- Rational Unified Process (RUP) is a commercial product that extends UP.
- It is a commercial version of UP from IBM
- It process icons map to the UP icons. → Fig
- UP is a modern SEP that is driven by user requirements and risk.
- UP has three basic axioms. It is
  - Requirements and risk driven;
  - Architecture-centric;
  - Iterative and incremental.



UP is an iterative and incremental process

- Large Software Development Project divides into mini projects.
- Each iteration contains *all* of the elements of a normal software development project:
  - Planning
  - Analysis and design
  - Construction
  - *I*ntegration and test
  - An internal or external release
- UP has five core workflows (Iteration workflows)
  - Requirements - capturing what the system should do;

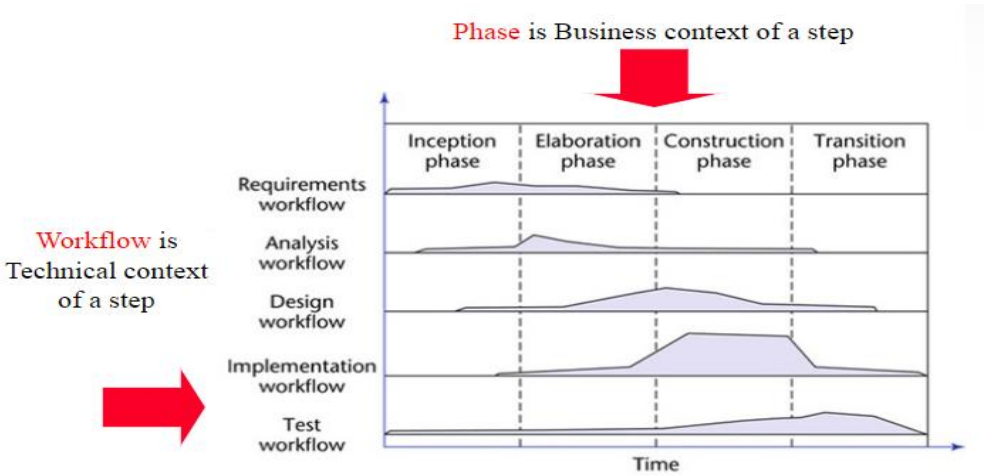
- Analysis - refining and structuring the requirements;
- Design - realizing the requirements in system architecture;
- Implementation - building the software;
- Test - verifying that the implementation works as desired.



- Breaking the project down into a series of iterations allows.
- Each iteration contain all of the five core workflows.
- Workflow depends on where the iteration occurs in the project life cycle.

UP structure

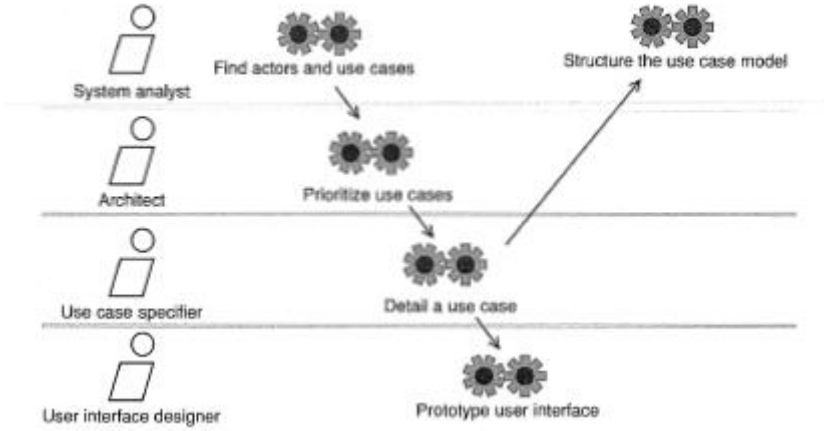
- The project life cycle is divided into four phases.
- Each of which terminates with a major milestone.
  - Inception : Life Cycle Objectives
  - Elaboration: Life Cycle Architecture;
  - Construction: Initial Operational Capability
  - Transition : Product Release.



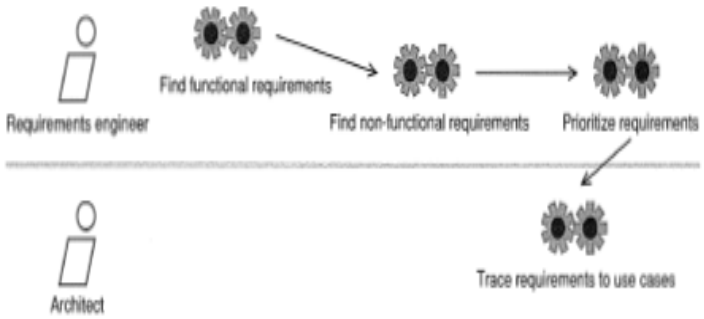
- Within each phase can have
  - One or more iterations, and
  - In each iteration execute the five core workflows and any extra workflows.
- The exact number of iterations per phase depends on the size of the project,

- One of the great features of UP is that it is a goal-based process rather than a deliverable-based process.
- Most requirements work is done at the beginning of the project in the Inception and Elaboration phases.

Requirement Workflow

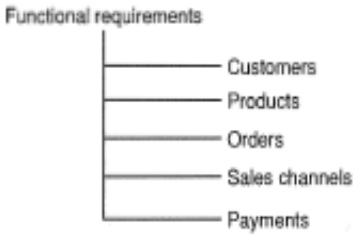


- A workflow detail shows us the workers and activities involved in a particular workflow.
- Make a simple extension to the UP requirements workflow to add the following new tasks.
  - Find functional requirements.
  - Find non-functional requirements.
  - Prioritize requirements.
  - Trace requirements to use cases.
- **Requirements** – Specifications of what should be implemented. (Basis)
- **Functional requirements** - what behavior the system should offer; (What the system should do)
- **Non-functional requirements** - a specific property or constraint on the system. (A Constraint on the System)
- In general Requirements will come from
  - Direct users of the system;
  - Other stakeholders (e.g., managers, maintainers, installers);
  - Other systems with which the system interacts;
  - Hardware devices with which the system interacts;
  - Legal and regulatory constraints;
  - Technical constraints;
  - Business goals.



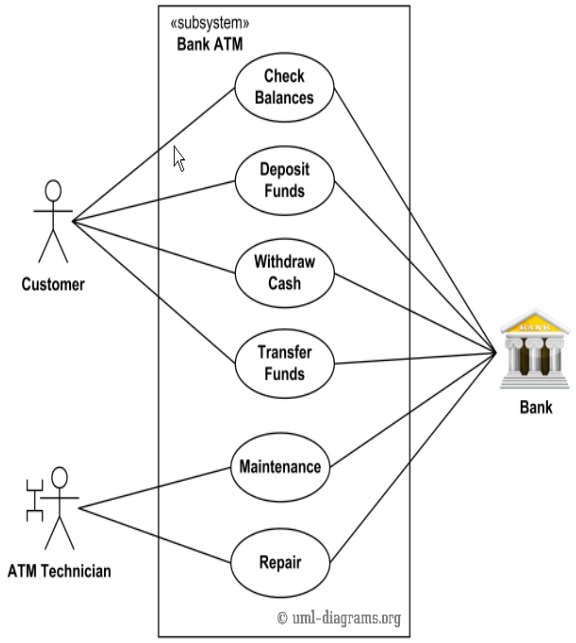
Case Study: ATM

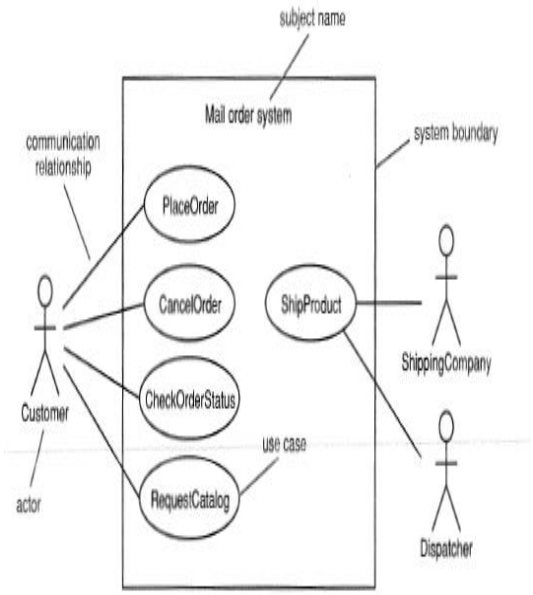
- Automated teller machine - ATM
- Functional requirements
  - The ATM system shall check the validity of the inserted ATM card.
  - The ATM system shall validate the PIN number entered by the customer.
  - The ATM system shall dispense no more than $250 against any ATM card in any 24-hour period.
- Non-functional requirements
  - The ATM system shall be written in C++.
  - The ATM system shall communicate with the bank by using 256-bit encryption.
  - The ATM system shall validate an ATM card in three seconds or less.
  - The ATM system shall validate a PIN in three seconds or less.



Use case modelling

- Use case modeling is a form of requirements engineering.
- It is the primary input to class modeling.
- It as follows
  - Find a candidate system boundary.
  - Find the actors.
  - Find the use cases:
    - specify the use case;
    - identify key alternative flows.
  - Iterate until use cases, actors, and system boundary are stable.





There are four components of this model.

- **System boundary** - A box drawn around the use cases to denote the edge or boundary of the system being modeled. This is known as the *subject* in UML 2.
- **Actors** - Roles played by people or things that use the system. (Externals)
- **Use cases** - Things that the actors can do with the system. (Point of view of Actor)
- **Relationships**. - Meaningful relationships between actors and use cases.

There is no UML standard for a use case specification.

- **Sample Template**
  - use case name;
  - use case ID;
  - Brief Description - a paragraph that captures the goal of the use case;
  - Actors involved in the use case;
  - Preconditions - these are things that must be true before the use case can execute-they are constraints on the state of the system;
  - Main Flow - the steps in the use case;
  - Postconditions - things that must be true at the end of the use case;
  - Alternative Flows - a list of alternatives to the main flow

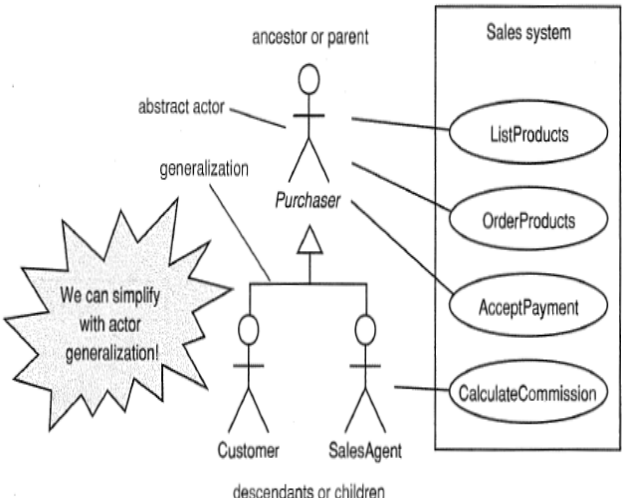
Use case: ManageBasket
ID: 2
Brief description: The Customer changes the quantity of an item in the basket.
Primary actors: Customer
Secondary actors: None.
Preconditions: 1. The shopping basket contents are visible.
Main flow: 1. The use case starts when the Customer selects an item in the basket. 2. If the Customer selects "delete item" 2.1 The system removes the item from the basket. 3. If the Customer types in a new quantity 3.1 The system updates the quantity of the item in the basket.
Postconditions: None.
Alternative flows: None.

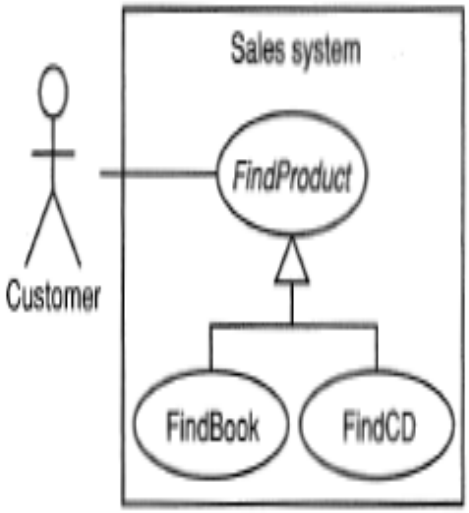
- Use case modeling is most appropriate for systems that:
  - are dominated by functional requirements;
  - have many types of user;
  - have many interfaces to other systems.
- Use case modeling is least appropriate for systems that:
  - are dominated by non-functional requirements;
  - have few users;
  - have few interfaces.

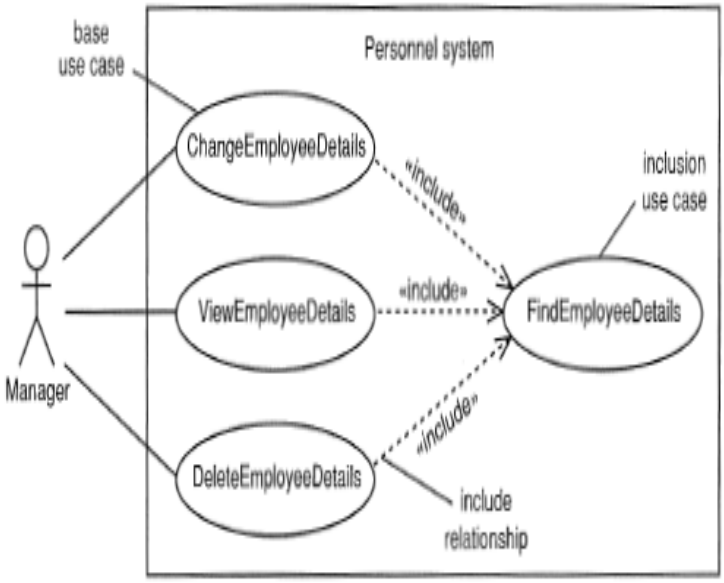
Advanced Usease Modeling

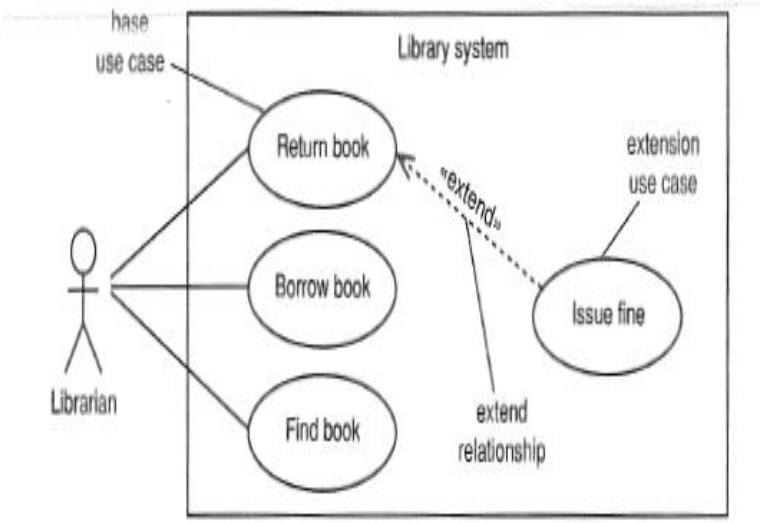
Use advanced features only when they simplify the model and make it easier to understand.

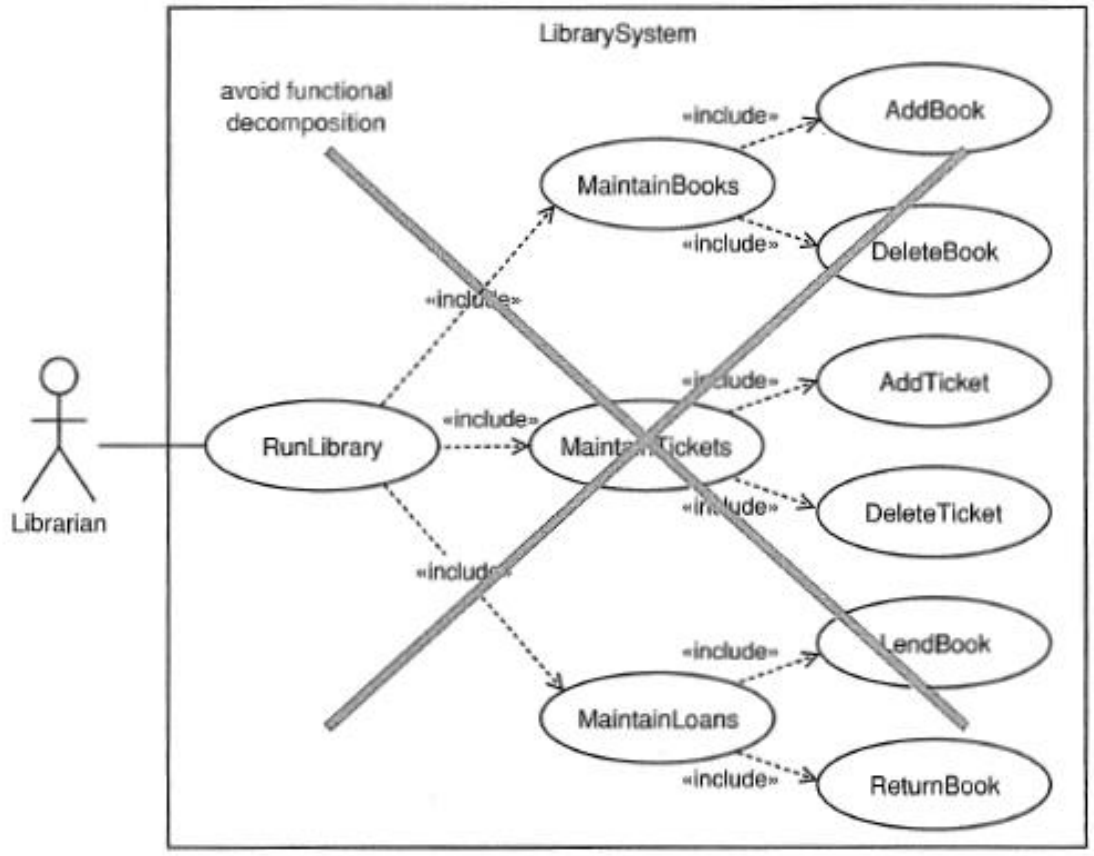
- **Actor generalization** - a generalization relationship between a more general actor and a more specific actor;
- **Use case generalization** - a generalization relationship between a more general use case and a more specific use case;
- **«include»** - a relationship between use cases that lets one use case include bbehaviour from another;
- **«extend»** - a relationship between use cases that lets one use case extend its behavior with one or more behavior fragments from another.


ancestor or parent
Sales system
abstract actor
ListProducts
generalization
Purchaser
OrderProducts
We can simplify with actor generalization!
AcceptPayment
CalculateCommission
Customer
SalesAgent
descendants or children


Sales system
FindProduct
Customer
FindBook
FindCD


base use case
Personnel system
ChangeEmployeeDetails
«include»
inclusion use case
ViewEmployeeDetails
«include»
FindEmployeeDetails
Manager
«include»
DeleteEmployeeDetails
include relationship


base use case
Library system
Return book
«extend»
extension use case
Borrow book
Issue fine
Librarian
Find book
extend relationship

Use case models- Avoid functional decomposition

- keep use cases short and simple;
- focus on the *what* not the *how;*
- avoid functional decomposition.